

Task1:

```
import heapq
```

```
graph = {  
    'S': [('A', 2), ('B', 5), ('C', 4)],  
    'A': [('D', 7), ('E', 3)],  
    'B': [('F', 6)],  
    'C': [('G', 2)],  
    'D': [('T', 4)],  
    'E': [('T', 6)],  
    'F': [('T', 5)],  
    'G': [('T', 3)],  
    'T': []  
}
```

```
def beam_search(start, goal, beam_width=2):  
    beam = [(0, [start])]   
  
    while beam:  
        candidates = []  
  
        for cost, path in beam:  
            current_node = path[-1]  
  
            if current_node == goal:  
                return path, cost  
  
            for neighbor, edge_cost in graph.get(current_node, []):
```

```
new_cost = cost + edge_cost
new_path = path + [neighbor]
candidates.append((new_cost, new_path))
```

```
beam = heapq.nsmallest(beam_width, candidates, key=lambda x: x[0])
```

```
return None, float('inf')
```

```
start_node = 'S'
```

```
goal_node = 'T'
```

```
for k in [1, 2, 3]:
```

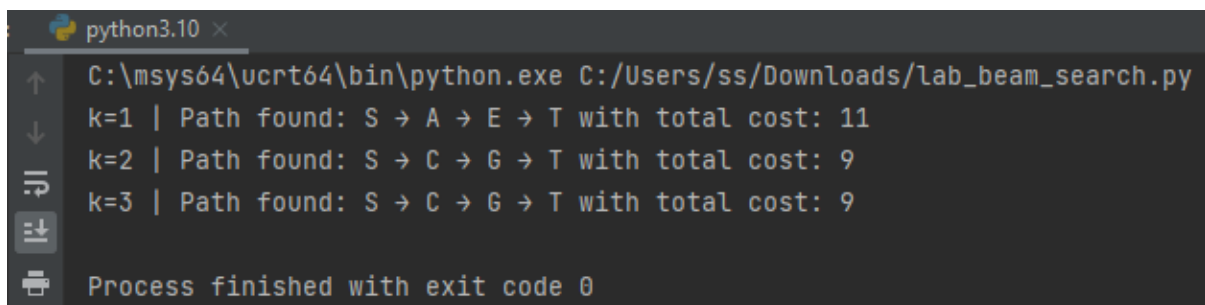
```
    path, cost = beam_search(start=start_node, goal=goal_node, beam_width=k)
```

```
    if path:
```

```
        print(f"k={k} | Path found: {' → '.join(path)} with total cost: {cost}")
```

```
    else:
```

```
        print(f"k={k} | No path found.")
```



```
python3.10 x
C:\msys64\ucrt64\bin\python.exe C:/Users/ss/Downloads/lab_beam_search.py
k=1 | Path found: S → A → E → T with total cost: 11
k=2 | Path found: S → C → G → T with total cost: 9
k=3 | Path found: S → C → G → T with total cost: 9
Process finished with exit code 0
```

Task2:

```
import random
```

```
def f(x): return -x*2 + 10x + 5
```

```
def simple_hill_climbing(): current_state = random.randint(0, 100) current_score =
f(current_state)
```

```
while True:
    neighbors = []
    if current_state - 1 >= 0:
        neighbors.append(current_state - 1)
    if current_state + 1 <= 100:
        neighbors.append(current_state + 1)

    next_state = None
    next_score = current_score

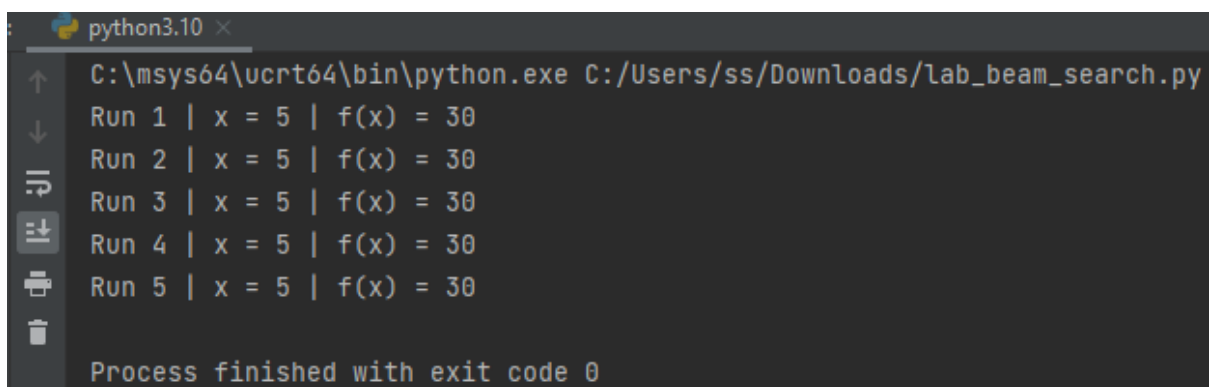
    for neighbor in neighbors:
        neighbor_score = f(neighbor)
        if neighbor_score > next_score:
            next_state = neighbor
            next_score = neighbor_score
            break

    if next_score <= current_score:
        break

    current_state = next_state
    current_score = next_score

return current_state, current_score

for i in range(5): x, score = simple_hill_climbing() print(f"Run {i+1} | x = {x} | f(x) = {score}")
```



```
python3.10 x
C:\msys64\ucrt64\bin\python.exe C:/Users/ss/Downloads/lab_beam_search.py
Run 1 | x = 5 | f(x) = 30
Run 2 | x = 5 | f(x) = 30
Run 3 | x = 5 | f(x) = 30
Run 4 | x = 5 | f(x) = 30
Run 5 | x = 5 | f(x) = 30
Process finished with exit code 0
```

Task3:

```
import random
```

```
cost_matrix = [
```

```
[4, 6, 8, 7, 5],  
[7, 5, 6, 8, 4],  
[6, 4, 7, 5, 8],  
[5, 8, 6, 4, 7],  
[8, 6, 5, 7, 4],  
[7, 4, 8, 6, 5],  
[6, 7, 4, 5, 8],  
[5, 6, 7, 8, 4],  
[4, 7, 5, 6, 8],  
[8, 5, 6, 4, 7]  
]
```

```
n_tasks = 10
```

```
n_machines = 5
```

```
mutation_rate = 0.1
```

```
def calculate_fitness(individual):
```

```
    total_cost = sum(cost_matrix[task][individual[task]] for task in range(n_tasks))
```

```
    return 1 / total_cost
```

```
def create_random_individual():
```

```
    return [random.randint(0, n_machines - 1) for _ in range(n_tasks)]
```

```
def select_parents(population, fitness_scores):
```

```
    sorted_population = [ind for _, ind in sorted(zip(fitness_scores, population),  
reverse=True)]
```

```
    return sorted_population[:len(population) // 2]
```

```
def crossover(parent1, parent2):
```

```
    point = random.randint(1, n_tasks - 2)
```

```
    child = parent1[:point] + parent2[point:]
```

```
    return child
```

```
def mutate(individual):
```

```
    idx1, idx2 = random.sample(range(n_tasks), 2)
```

```
    individual[idx1], individual[idx2] = individual[idx2], individual[idx1]
```

```
    return individual
```

```
def genetic_algorithm(population_size):
```

```
    population = [create_random_individual() for _ in range(population_size)]
```

```
    best_fitness = 0
```

```
    best_individual = None
```

```
    best_generation = 0
```

```
    for generation in range(100):
```

```
        fitness_scores = [calculate_fitness(ind) for ind in population]
```

```
        current_best_fitness = max(fitness_scores)
```

```
        if current_best_fitness > best_fitness:
```

```
            best_fitness = current_best_fitness
```

```
            best_individual = population[fitness_scores.index(current_best_fitness)]
```

```
            best_generation = generation + 1
```

```
    if best_fitness == 1.0:
```

```
        break
```

```
parents = select_parents(population, fitness_scores)

new_population = [crossover(random.choice(parents), random.choice(parents)) for
_ in range(population_size)]

for i in range(len(new_population)):
    if random.random() < mutation_rate:
        new_population[i] = mutate(new_population[i])

population = new_population

return best_individual, calculate_fitness(best_individual), best_generation

for pop_size in [10, 30]:
    print(f"\nPopulation Size: {pop_size}")
    print("-" * 40)
    individual, fitness, generation = genetic_algorithm(pop_size)
    total_cost = sum(cost_matrix[task][individual[task]] for task in range(n_tasks))
    print(f"Best Chromosome : {individual}")
    print(f"Total Cost    : {total_cost}")
    print(f"Fitness      : {fitness:.4f}")
    print(f"Best Generation : {generation}")
```

```
python3.10 x
C:\msys64\ucrt64\bin\python.exe C:/Users/ss/Downloads/lab_beam_search.py

Population Size: 10
-----
Best Chromosome : [4, 4, 3, 3, 4, 4, 2, 4, 0, 2]
Total Cost      : 45
Fitness         : 0.0222
Best Generation : 35

Population Size: 30
-----
Best Chromosome : [0, 4, 1, 3, 4, 1, 2, 4, 0, 3]
Total Cost      : 40
Fitness         : 0.0250
Best Generation : 10

Process finished with exit code 0
```